

Day 4 – Your First Lightning Web Component (LWC)

Part 1 – Think Before Coding

1. Why do users need a graphical interface?

Users need a graphical user interface (GUI) because it allows them to interact with Salesforce easily without knowing programming or database commands.

For example, users can click buttons, fill forms, view tables, and search records instead of writing code.

2. Why can't users directly execute SOQL queries?

Normal users should not execute SOQL queries directly because:

- They may not know SOQL syntax.
- Incorrect queries can cause errors or poor performance.
- Direct querying is not user-friendly.
- Data access must follow Salesforce security and business rules.

Instead, the application retrieves the required data and displays it through the UI.

3. Why is JavaScript required in LWC?

JavaScript is required in Lightning Web Components (LWC) to control the component's logic and behavior.

It is used to:

- Handle button clicks and user actions.
- Store and process data.
- Call Apex methods.
- Validate user input.
- Update the UI dynamically.

HTML = Structure/UI

JavaScript = Logic/Behavior

4. What responsibilities belong to the UI?

The UI should mainly handle user interaction and presentation, such as:

- Displaying records and information.
- Taking input from users.
- Showing buttons, forms, tables, and messages.
- Performing basic client-side validation.

- Sending user requests to Apex when necessary.
- Displaying success and error messages.

5. Which responsibilities should remain in Apex?

Apex should handle server-side and business logic, especially when the logic is complex or requires database operations.

Apex responsibilities include:

- Executing SOQL queries.
- Performing DML operations such as insert, update, and delete.
- Implementing complex business rules.
- Processing large amounts of data.
- Handling transactions.
- Enforcing appropriate security and data-access logic.
- Providing data to LWC components.

Part 2 – Understanding LWC

Every Lightning Web Component (LWC) consists of three important files:

1. studentHome.html – HTML

The HTML file is responsible for the user interface and page structure.

It handles:

- Page layout
- Buttons
- Text
- Images
- Input fields
- Displaying data

Example: Creating a Student Home Page with a heading, student details, and buttons.

2. studentHome.js – JavaScript

The JavaScript file is responsible for the logic and behavior of the component.

It handles:

- Logic
- Variables

- Events
- Button-click actions
- User input
- Communication with Apex
- Processing data

Example: When the user clicks View Applications, JavaScript handles the action.

3. studentHome.js-meta.xml – Meta XML

The Meta XML file defines where and how the LWC can be used in Salesforce.

It is responsible for:

- Making the component available in Lightning App Builder
- Defining where the component can be used
- Exposing the component to Lightning pages
- Setting targets such as App Page, Home Page, and Record Page

Easy Way to Remember

File	Responsibility
studentHome.html	What the user sees
studentHome.js	What the component does
studentHome.js-meta.xml	Where the component can be used

Part 3 – Hands-on Activity 1

Objective

Create the first Lightning Web Component (LWC) for the Placement Management System and deploy it on a Lightning Page.

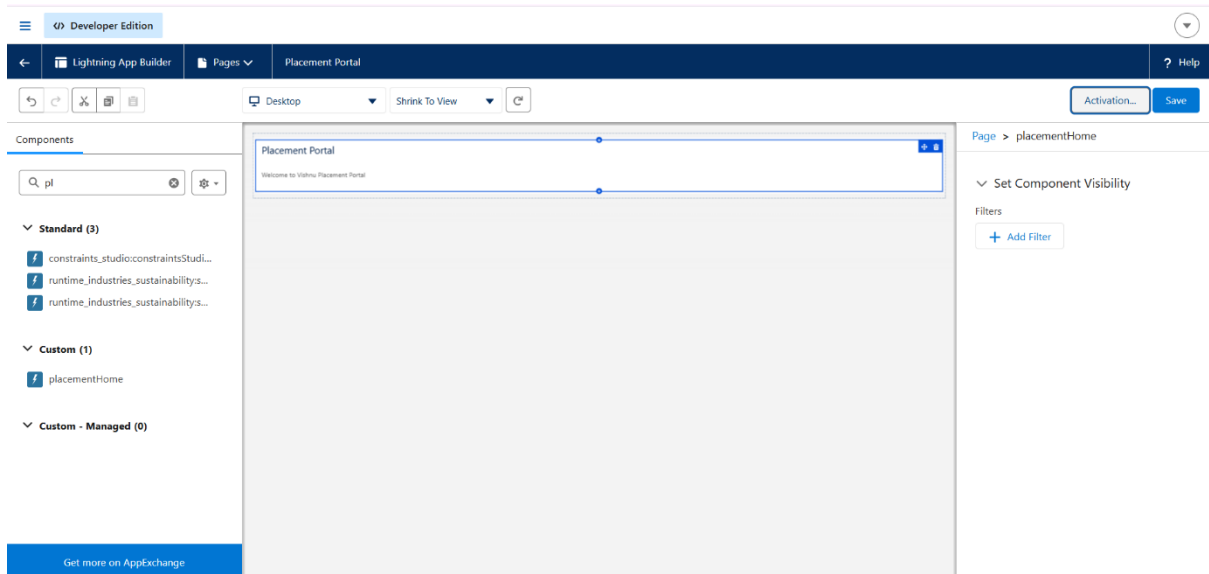
Component Name

placementHome

Output

Placement Portal

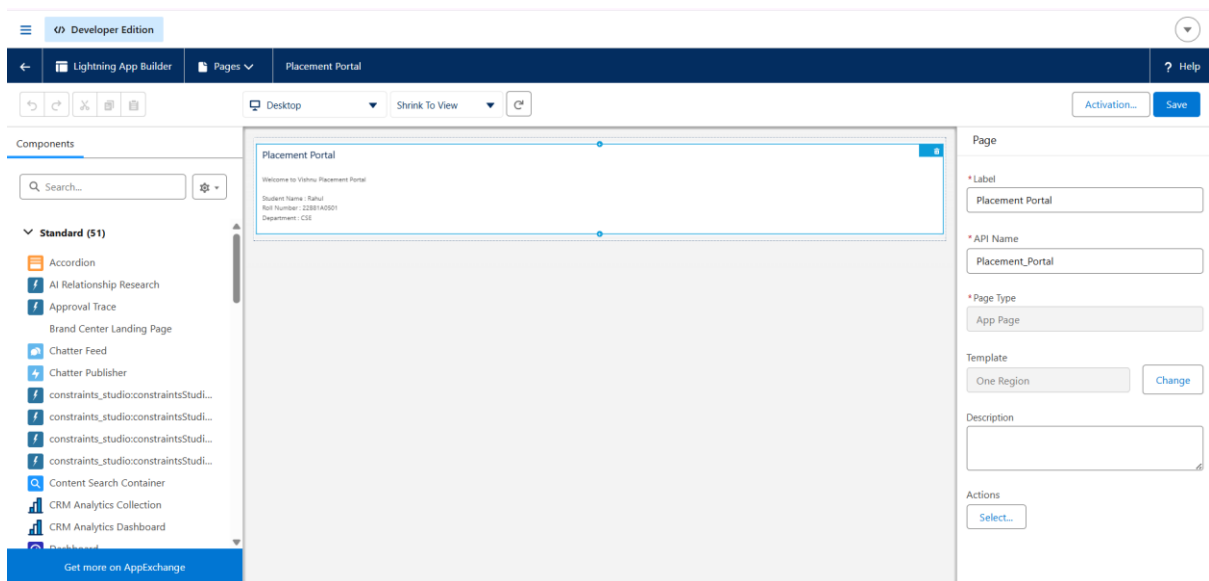
Welcome to Vishnu Placement Portal.



Part 4 – Hands-on Activity 2

Objective

To create Student Name, Roll Number, and Department variables in an LWC, display their values on the screen, and observe how the UI changes when the variable values are updated.



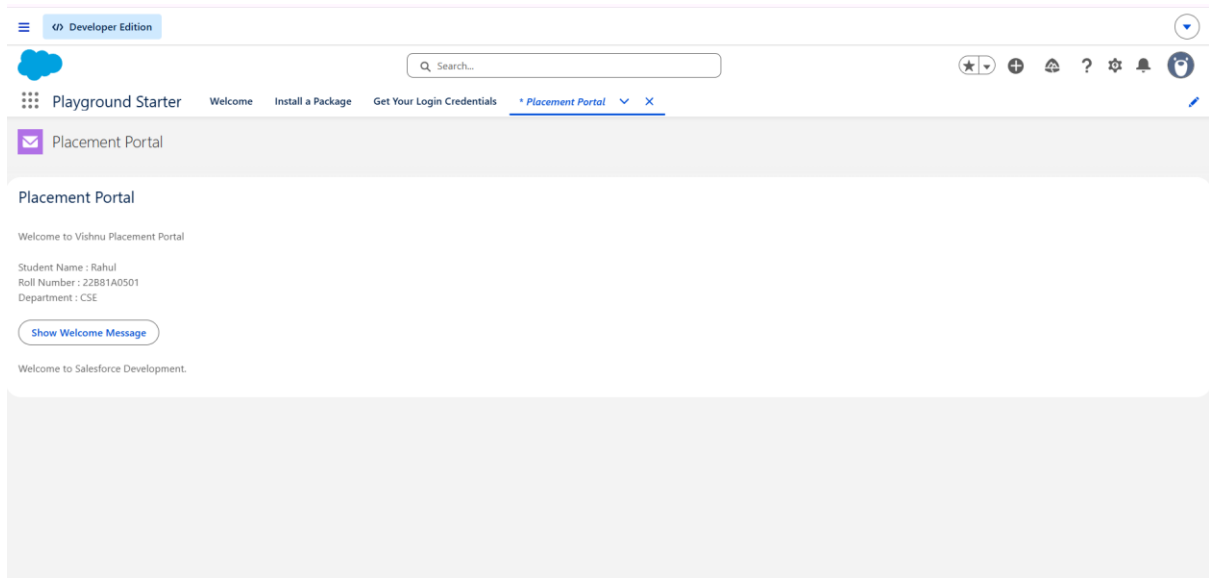
Part 5 – Hands-on Activity 3

Objective

To create a button in an LWC, handle a click event using JavaScript, and display a welcome message dynamically.

Initially, only the Show Welcome Message button appears.

After clicking the button, it displays: Welcome to Salesforce Development.



Part 6 – Hands-on Activity 4

Objective

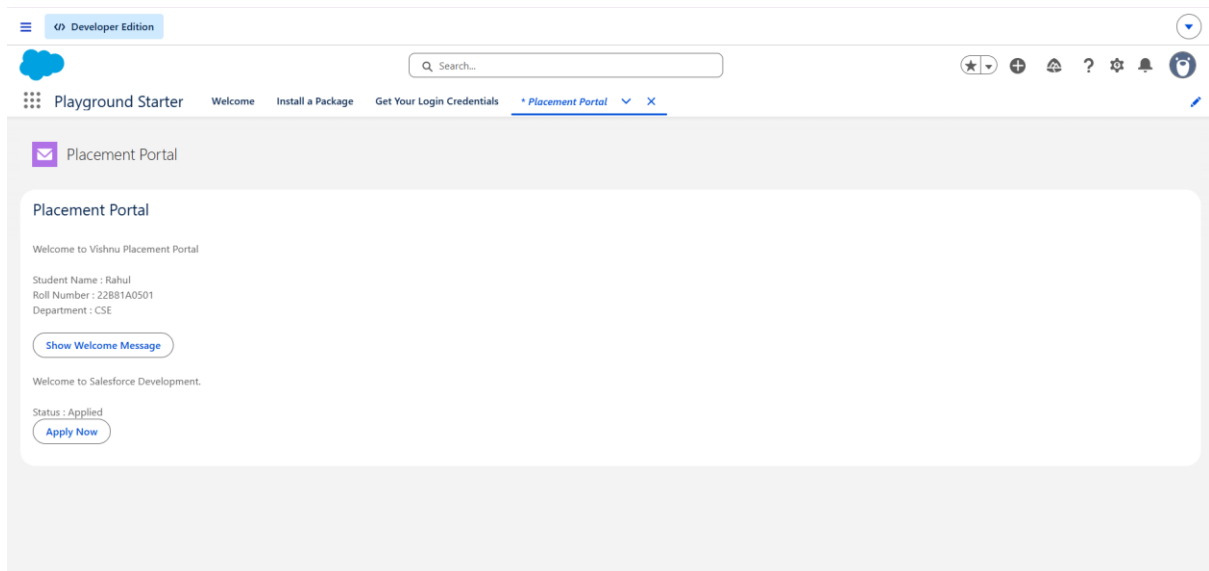
To understand how JavaScript can dynamically change data in an LWC without using Apex or a database.

Before clicking the button:

Status : **Not Applied**

After clicking Apply

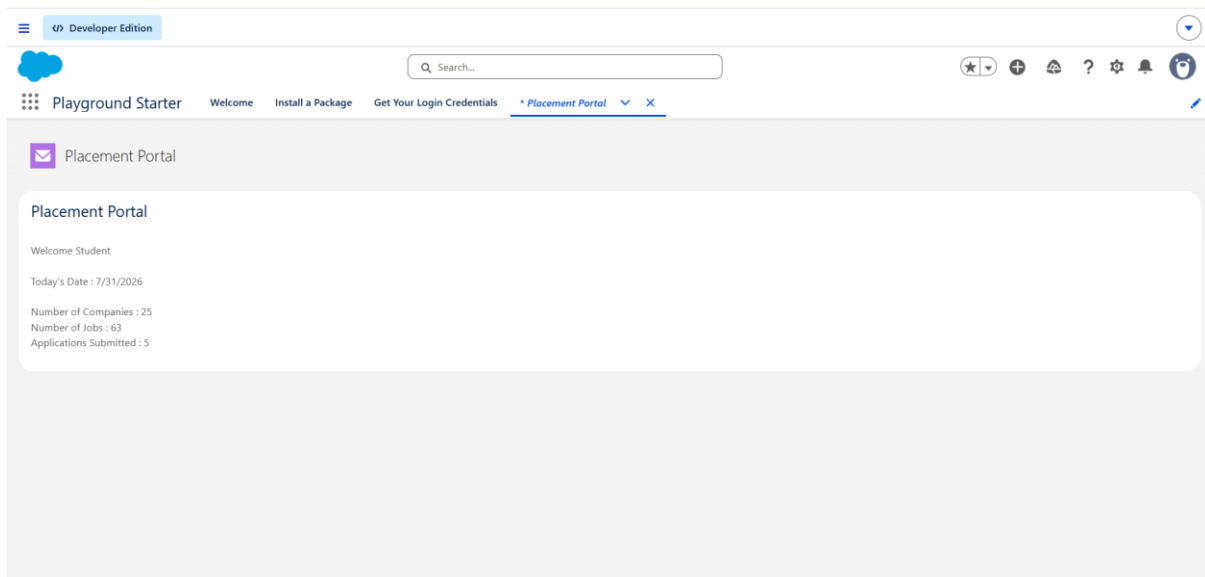
Status : **Applied**



Part 7 – Mini Project Enhancement

Objective

To create the first screen of the Placement Management System using LWC and display placement information using hard-coded JavaScript values.



Part 8 – Understanding Data Binding

Objective

To understand data binding in LWC by displaying a JavaScript variable in HTML and observing how the UI changes when the variable value is changed.

